

Part 1

How LLMs Work

Practical intuition, not research math

Tokens: The Unit of Text

"Refactor the checkout service and run the tests."

Refactor

the

check

out

service

and

run

the

tests

.

The model does not see words exactly the way humans do. It sees token IDs, then predicts what token should come next.

Next Token Prediction

PROMPT

The API request failed
because the auth token
was

expired  46%

missing  24%

invalid  18%

rotated  8%

Generation is repeated prediction: choose a token, append it, predict again.

The Transformer Changed the Scaling Curve

Before

Models struggled to use long context well.

2017

Attention

Tokens can directly weigh other relevant tokens.

After

Large context, parallel training, stronger language behavior.

The important idea for developers: attention lets the model decide which parts of the context matter for the next step.

Vector Space: Meaning Becomes Geometry



This is a simplification. The actual space has many dimensions, but the intuition is useful: related concepts end up near each other.

Why Answers Are Not Always Identical

expired

missing

invalid

rotated

Low randomness

More consistent, more conservative, better for exact tasks.

Higher randomness

More variation, more exploration, useful for ideation.

Creativity is often controlled by sampling strategy, not by a separate imagination module.

Thinking Models

Basic generation

Predict the next visible token.



Reasoning behavior

Spend more internal compute before answering or acting.

For practical use, reasoning models do more internal work before responding: understand the task first, explore possible paths, check for mistakes, and revise before showing the final answer.

The Developer Mental Model

Context shapes behavior

The model can only use what it can see.

Prediction is not correctness

A plausible answer still needs verification.

Tools change capability

A model with tests, logs, and file access is a different system.

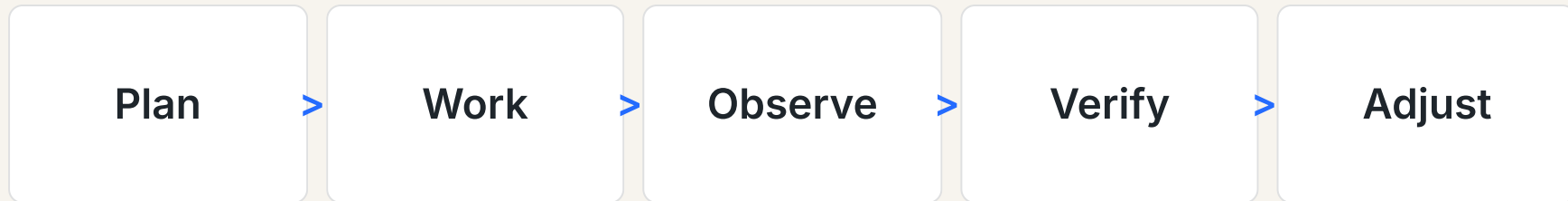
AI works unusually well for coding because code has stricter semantics than ordinary language: APIs, types, tests, compilers, and runtime errors give the model clearer structure and faster feedback.

Part 2

AI Agents

A model inside a work loop

Agent Loop



An agent is useful because it can keep moving: make a change, inspect the result, correct course, and repeat.

Work

Read files

Edit code

Search docs

Run commands

Call APIs

Open browser

The model provides judgment. The tools provide contact with the actual system.

Verify

Tests

Does behavior still hold?

Lint

Does the code meet local rules?

Logs

What actually happened?

Screenshots

Does the UI render correctly?

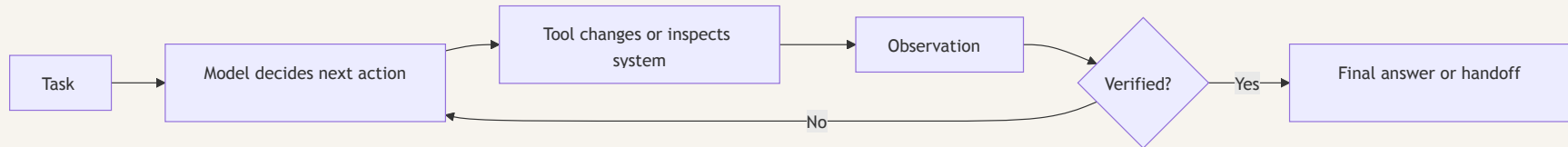
Types

Did contracts break?

Diffs

What changed?

Loop Until Done



The loop matters more than any single prompt. The agent improves by staying connected to feedback.

Part 3

Harness

Guardrails that keep the model on path

Raw Model vs Harnessed System

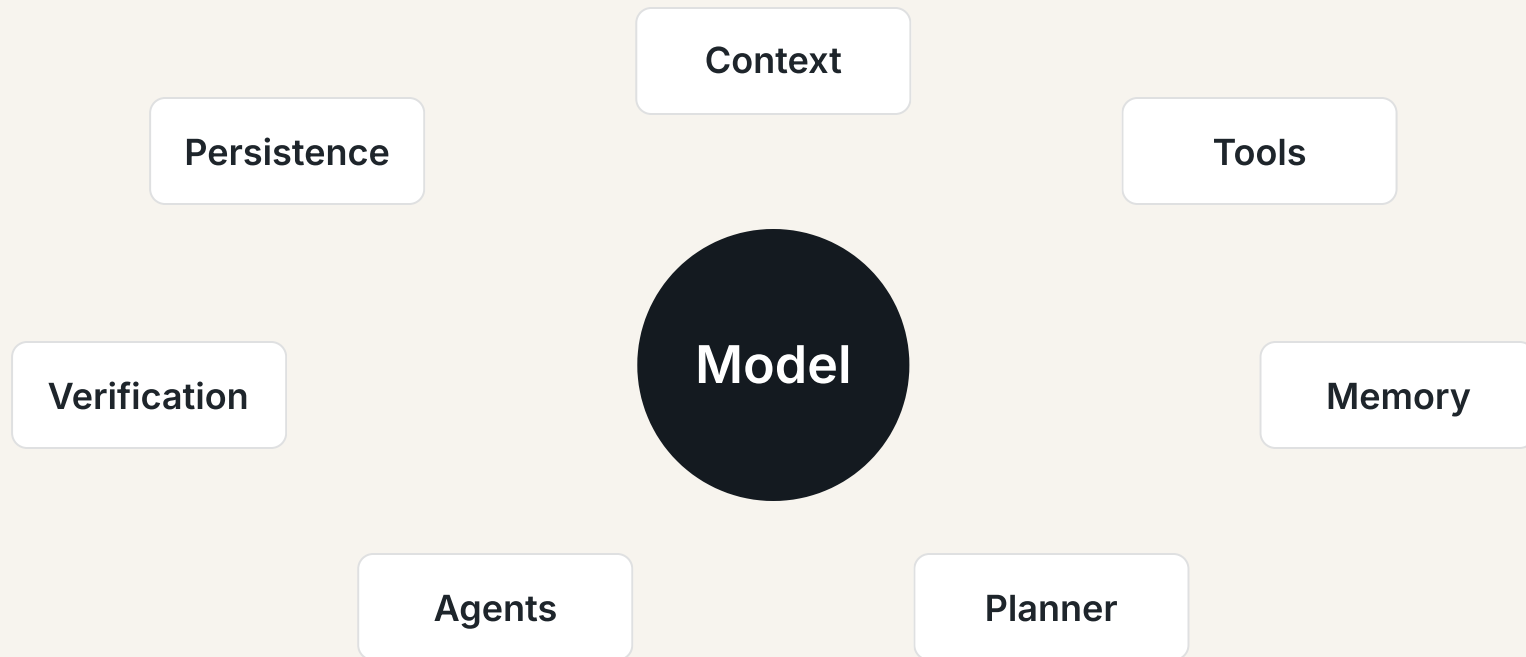
Raw model

- Answers from prompt context
- May invent missing state
- Cannot inspect the real system
- Correctness is mostly user-reviewed

Harnessed system

- Reads project context
- Uses tools and logs
- Runs verification
- Persists progress and constraints

Harness Components



Context Management

Compression

Summarize older work so important state survives limited context.

Prioritization

Keep the most relevant instructions, files, and evidence near the current task.

Expansion

Fetch missing project context, docs, logs, tests, or examples when needed.

Context management is the harness continuously shaping what the model sees, so the next answer is grounded in the right task, evidence, and constraints.

Tools

filesystem

terminal

git

browser

tests

database

docs

CI

Tool access turns language ability into engineering ability.

Memory, Planning, Persistence

Memory

Combines short-term session history with recorded long-term information the system can bring back later.

Planner

Creates a task list the model can follow step by step, keeping work focused instead of drifting.

Persistence

Stores memories, session state, and progress so work can continue across multiple runs.

Verification Is the Control Surface

01 Logs, diffs, screenshots

02 Unit tests and types

03 Integration tests

04 Product behavior

05 Human judgment

AI does not remove engineering discipline. It makes disciplined feedback loops more valuable.

Part 4

AI Coding Evolution

From autocomplete to teams of agents

The Evolution

1

AI Autocomplete

Suggest the next line.

2

Editor Agent

Edit within the IDE.

3

CLI Agent

Operate in the project environment.

4

Specialized Agent

Own a workflow with tools and verification.

5

Multi-Agent

Parallel specialists with coordination.

What Changes for the Team

BEFORE

Developer writes every step

Manual implementation is the main unit of progress.

AFTER

Developer designs the work system

Tasks, context, review, and verification become the main leverage points.

New Team Muscles

Task design

Define work so an agent can execute it.

Context packaging

Give the model the right project evidence.

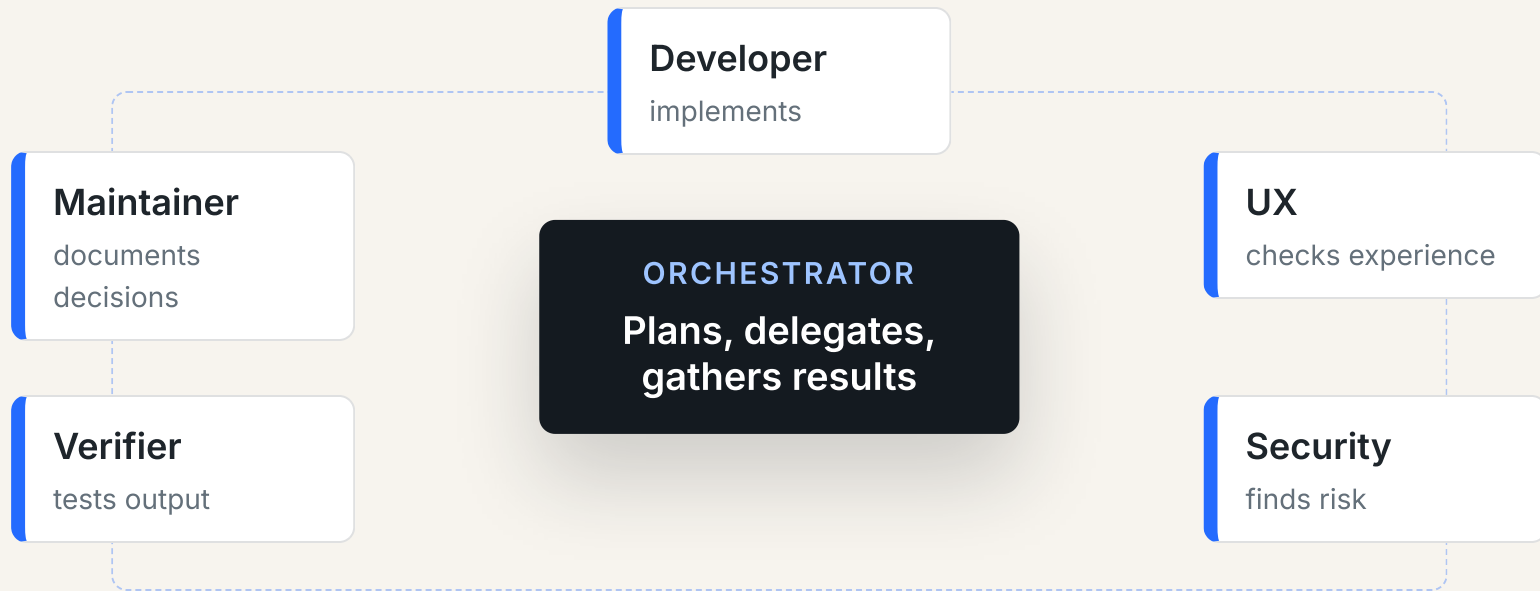
Review discipline

Inspect diffs, behavior, and assumptions.

Verification ownership

Make correctness observable and repeatable.

Multi-Agent Future



The future is not one model doing everything. It is coordinated specialists with clear roles, shared context, and verification between handoffs.

Part 5

Agentic Engineering
Practical steps for real projects

From Prompting To Engineering

AD HOC

Prompt the model

One-off requests depend heavily on what the developer remembers to include.



ENGINEERED

Shape the work system

Specs, rules, tools, tests, and review loops make AI work repeatable.

The team's leverage moves from clever prompts to reusable project context and reliable feedback loops.

Specs Become Context

Project spec

What the system is, who it serves, and how major parts fit together.

Module specs

Boundaries, responsibilities, data contracts, edge cases, and ownership.

Task specs

The current change, acceptance criteria, risks, and expected verification.

Good specs reduce repeated codebase search and give each new AI session a stable starting point.

Rules Shape The Agent

`RULES.md`

Team preferences, coding style, architecture boundaries.

`AGENTS.md`

How agents should work in this repo: commands, checks, handoff rules.

`module docs`

Local conventions and constraints near the code they govern.

The model should not rediscover team taste and repo conventions on every task. Write them down where agents can load them.

Smart Verification

Fast loop

Types, focused unit tests, lint, targeted logs.

Risk loop

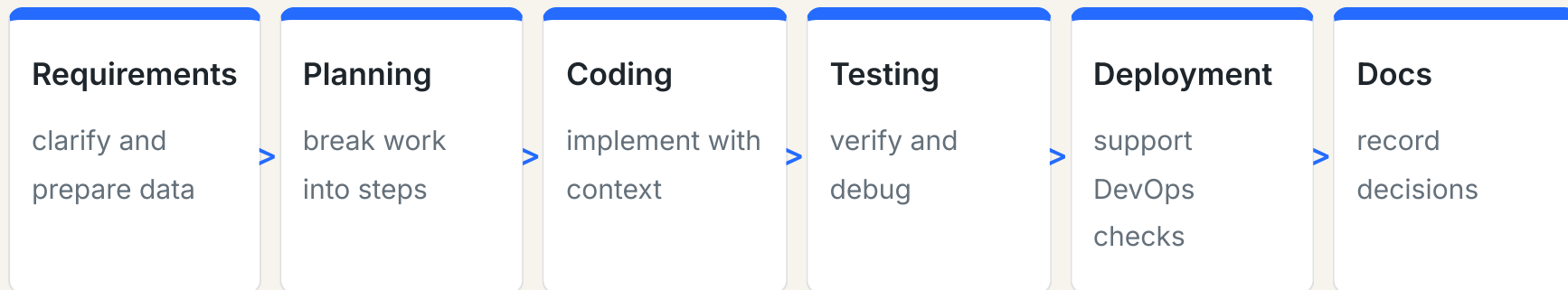
Critical paths, edge cases, integration points, screenshots.

Release loop

Full suite, CI, deployment checks, human review.

Do not test everything on every iteration. Test the critical parts early, then widen verification when confidence matters.

AI Across The SDLC



This is not a rigid lifecycle. It is a reminder that AI can help across the process when each stage has the right context and checks.

Agentic Features

Skills

Package repeatable workflows and domain knowledge.

MCP

Connect agents to tools, data, systems, and services.

Commands

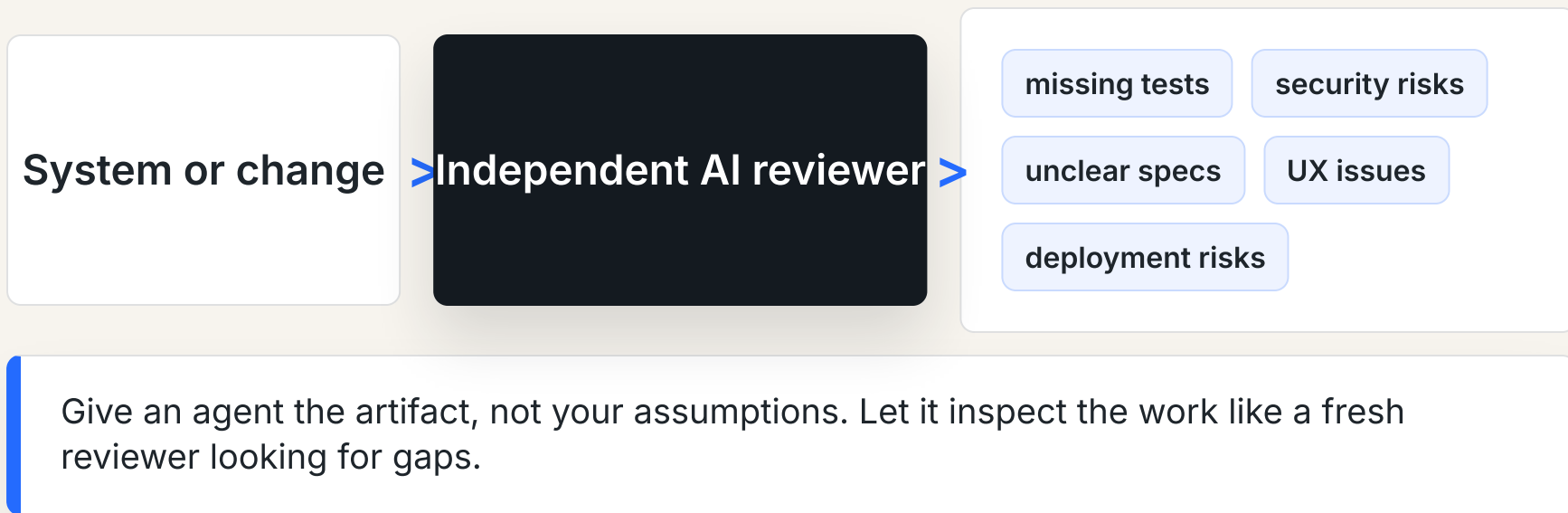
Turn common repo operations into reliable actions.

Sub-agents

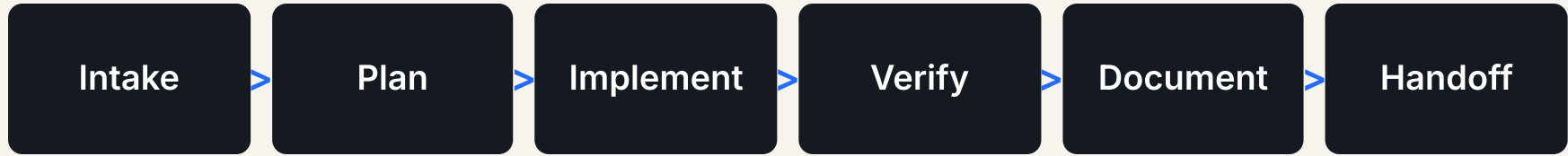
Delegate bounded work to specialists in parallel.

These features matter because they turn individual AI help into reusable team capability.

Blind-Box Auditing



Workflow Automation



The direction is clear, but the standard practice is still forming: teams will gradually automate larger parts of the loop while keeping human ownership at checkpoints.